

## Lab Program 03: Implement 8 Puzzle using DFS & Iterative DLS

# IDLS

```
GOAL_STATE = (1, 2, 3,  
              8, 0, 4,  
              7, 6, 5)
```

```
INITIAL_STATE = (2, 8, 3,  
                1, 6, 4,  
                7, 0, 5)
```

```
def get_neighbors(state):  
    neighbors = []  
    idx = state.index(0)  
    row, col = idx // 3, idx % 3  
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]: # Up, Down, Left, Right  
        nr, nc = row + dr, col + dc  
        if 0 <= nr < 3 and 0 <= nc < 3:  
            new_idx = nr * 3 + nc  
            lst = list(state)  
            lst[idx], lst[new_idx] = lst[new_idx], lst[idx]  
            neighbors.append(tuple(lst))  
    return neighbors
```

```
def print_row_of_grids(states, label=""):  
    """Prints multiple 3x3 grids horizontally in a clean single row"""
```

```

if not states: return

print(label)

for r in range(3):

    row_str = []

    for state in states:

        grid_row = [str(x) if x != 0 else '_' for x in state[r*3:(r+1)*3]]

        row_str.append(" ".join(grid_row))

    print(" | ".join(row_str))

print()

```

```

def depth_limited_dfs(current_state, goal_state, limit, path, visited):

```

```

    if current_state == goal_state:

        return path

```

```

    if limit <= 0:

        return None

```

```

    branches = get_neighbors(current_state)

    valid_branches = [b for b in branches if b not in visited]

```

```

    print_row_of_grids([current_state], f"Parent (Remaining Depth Limit: {limit}):")

    print_row_of_grids(valid_branches, "Possibilities:")

```

```
print("-" * 45)
```

```
for b in valid_branches:
```

```
    visited.add(b)
```

```
    result = depth_limited_dfs(b, goal_state, limit - 1, path + [b], visited)
```

```
    if result is not None:
```

```
        return result
```

```
    visited.remove(b)
```

```
return None
```

```
def run_ids(initial_state, goal_state, max_depth_limit):
```

```
    for depth in range(max_depth_limit + 1):
```

```
        print(f"\nTRYING DEPTH LIMIT: {depth}")
```

```
        visited = {initial_state}
```

```
        solution = depth_limited_dfs(initial_state, goal_state, depth, [initial_state], visited)
```

```
        if solution is not None:
```

```
            print("\nGOAL REACHED!")
```

```
            print(f"Optimal solution path found at depth limit {depth}:")
```

```
            for index, state in enumerate(solution):
```

```
                print(f"Step {index}:")
```

```
                print_row_of_grids([state])
```

```
return
```

```
print(f"\nSearch complete. No solution found within a depth limit of {max_depth_limit}.")
```

```
if __name__ == "__main__":
```

```
    try:
```

```
        user_limit = int(input("Enter maximum depth limit for Iterative Deepening Search: "))
```

```
        run_ids(INITIAL_STATE, GOAL_STATE, user_limit)
```

```
    except ValueError:
```

```
        print("Please enter a valid integer.")
```

## Output:

```
1 0 4
7 6 5

Possibilities:
1 2 3
0 4
7 6 5

-----
Parent (Remaining Depth Limit: 1):
1 2 3
0 4
7 6 5

Possibilities:
1 2 3 | 1 2 3
7 0 4 | 0 4
6 5 | 7 6 5

-----

GOAL REACHED!
Optimal solution path found at depth limit 5:
Step 0:
2 0 3
1 6 4
7 6 5

Step 1:
2 0 3
1 6 4
7 6 5

Step 2:
2 0 3
1 6 4
7 6 5

Step 3:
1 2 3
0 4
7 6 5

Step 4:
1 2 3
0 4
7 6 5

Step 5:
1 2 3
0 4
7 6 5

>>>
```

Ln: 475 Col: 0